Development of Battlemage Arena

1st Toke Odgaard-Jans
toodg22@student.sdu.dk

2nd Marcus Elmkvist
maelm22@student.sdu.dk

3rd Oliver
owagn20@student.sdu.dk

4th Mansoor
moahm25@student.sdu.dk

I. Introduction

The goal of this project is to create a multiplayer game centered around physics-based interactions. The game takes place in a magical tournament where two players compete in an arena using physics-driven spells.

Unlike traditional combat systems, players cannot directly damage each other. Instead, damage occurs indirectly through environmental interactions. The damage calculation is based on physical forces, heavier and faster objects cause more damage.

Spells were supposed to interact dynamically with the environment, allowing creative combinations for unique effects. Players were also supposed to take damage from traps or hazards within the arena. To enhance realism and fun, ragdoll physics is implemented for player characters.

II. Game design

A. Overview

Battle Mage Arena is a multiplayer game where players control mages in an arena and use physics-based spells to manipulate the environment. The core mechanic is indirect damage: players cannot harm each other directly but must use spells to create environmental hazards that deal damage through physical interactions.

B. Main Mechanics

- **Physics-Based Combat:** Spells do not deal direct damage. Instead, they apply forces to objects, creating dynamic interactions. Damage is calculated based on impact force using the simplified formula:

$$\text{Force} = m \times v \tag{1}$$

  where m is the object's mass and v its velocity. Heavier and faster objects cause more damage.
- **Spells:**
  - **Force Spell:** Applies an impulse to objects via a raycast, pushing them in the opposite direction of the cast.
  - **Ice Spell:** Creates an ice surface that removes friction for actors in contact, enabling creative movement and trap setups.
  - **Mass Increase Spell:** Upon hitting a synchronized object with a raycast, the objects mass and size are both increased.
- **Health and Ragdoll:** When a player is hit with sufficient force, they take damage and enter ragdoll mode for a short cooldown period.
- **Multiplayer Synchronization:** The game uses Unreal Engine's listen server model. Positions, velocities, and spell effects are synchronized across clients to maintain consistent physics interactions.
- **Game Modes:** Multiple game modes are supported through a parent-child blueprint structure, allowing easy extension and customization.
- **HUD and Feedback:** Players have a HUD displaying health, score, and a timer. Audio feedback includes spell sounds.
- **Levels and level Selector:** The games has three levels
  - **Arena Battle:** A small arena filled with synchronized physics boxes. The goal is to eliminate the other player by hitting them with a box.
  - **Soccer:** A small soccer arena with a ball in the middle, the goal is to get the ball into the other players' goal. First to 5 wins.
  - **Coin Collection:** Small arena filled with synchronized physics boxes. Coins spawn at random points within the level. The goal is to collect 5 coins to win. If hit by a physics box you will lose coins.

III. Implementation

A. Damage

B. Ragdoll

C. Force Push

D. Healthbar

E. Ice Spell

F. Mass Increase

IV. Development

Xxxxx

A. Iteration 1

The first iteration was focussed on setting up the project and implementing some basic online functionality.

1) About: The purpose of this iteration was to setup the project and implement basic functionality for testing purposes. We needed to install Unreal Engine, ensure all the team members could work on the project, and set up version control using git. Character movement and physics manipulation should be synchronized across client and server. As such, this iteration also concerned learning about the Unreal Engine 5 client/server model, synchronizing player movement, and implementing a physics object which had its location and physics properties synchronized across client and server.

We also implemented a force spell that pushes the object in a direction by applying a force to it.

Finally, we implemented a simple health system for the player, so that colliding with the box would result in the player taking damage.

2) Evaluation: Since the goal of this iteration was to test if the technical aspects of the game worked, testing was done internally in the group. After a feature had been developed, it would be playtested on two connected clients that were simulated by the engine. The success of this iteration was judged to the extent that the synchronized objects behaved as expected.

We used the built-in systems for synchronization and online play to synchronize the box, the player, and player health. Making it so the player took damage when hit by the box was more challenging. The game only registered a hit when the player walked into the box, not when the box was thrown at the player.

3) Reflections and decisions: Most systems worked well. For the next iteration it was agreed that we would get the player to take damage when hit by physics-affected objects. Otherwise, the next iteration would be focussed on adding new features to the game.

B. Iteration 2

This iteration concerned getting the game ready for user testing.

1) About: Xxxxx
2) Evaluation: Xxxxx
3) Reflections and decisions: Xxxxx

C. Iteration 3

Xxxxx
1) About: Xxxxx
2) Evaluation: Xxxxx
3) Reflections and decisions: Xxxxx

D. Final iteration

Xxxxx

V. Reflection

Throughout this project, we have successfully developed the necessary mechanics and functionalities to effectively showcase the concept of the game. However, with additional time, there are several mechanics we would consider implementing to enhance gameplay and player experience.

1) **Environmental Hazards and Enhanced Maps:** The current game maps are somewhat simplistic in both aesthetics and functionality. Given more time for development, we could introduce environmental hazards that create dynamic challenges for players, enhancing the overall gameplay experience.
2) **Destruction of the Environment:** Enhancing the environment's interactivity through destructible elements would add depth. For instance, when a player uses a spell or interacts with an object (like a pillar), the environment could react by collapsing, with debris functioning as synchronized damage objects. This would not only increase visual appeal but also introduce strategic elements to gameplay as players navigate the altered terrain.
3) **Expanded Spell Variety:** Currently, the game only features three spells, limiting the strategic interactions between players. By adding more spells, we could create opportunities for more complex interactions. For example, spells could be designed to spawn additional projectiles for players to use against opponents or create new environmental hazards. If fall damage were to be introduced, a spell that propels an opponent into the air could create thrilling moments in the game.
4) **Improved Controls and Movement Mechanics:** At present, player movement is limited to basic walking and jumping. Particularly in the soccer game mode, a running mechanic would significantly enhance player control and positioning, allowing for a more immersive gameplay experience.

Incorporating these mechanics would not only improve the visual and strategic elements of the game but also elevate the user experience by making the gameplay more engaging and interactive.